

Design decisions

Dependencies

No additional direct dependencies were used outside the allowed web framework (Gin) and GORM with its PostgreSQL driver.

Part 1

- URL normalization:

Two URLs are considered the same after normalization.

The normalization process:

- Removes surrounding whitespace.
- Converts the URL scheme and host to lowercase.
- Removes default ports:
 - :80 for HTTP.
 - :443 for HTTPS.
- Validates that only `http` and `https` schemes are accepted.

Paths, query parameters, and fragments are preserved because they may represent different resources.

- Same URL → same code:

The application checks whether a normalized URL already exists before generating a new code.

The repository stores both:

- normalized long URL
- generated short code

If the URL already exists, the existing code is returned.

- Code generation:

New URLs receive a randomly generated 6-character alphanumeric code.

Codes contain only:

- lowercase letters
- uppercase letters
- numbers

Code generation happens only for URLs that do not already exist.

- Collision handling:

Generated codes are checked through the repository before being returned.

If a collision occurs, generation is retried until a unique code is found.

- Mutex and locking:

The in-memory repository uses `sync.RWMutex`.

Lock usage:

- `Save()` uses `Lock()` because it modifies shared state.
- `FindByCode()` uses `RLock()` because it only reads data.
- `FindByURL()` uses `RLock()` because it only reads data.

RWMutex was chosen because redirect and lookup operations are expected to occur more frequently than writes.

- Package layout:

The project follows a Clean Architecture style:

- `domain`
 - Entities
 - Domain errors
 - Repository interfaces
- `application`
 - Use cases
 - DTOs
 - Business rules
- `infrastructure`
 - Database connection
 - Repository implementations
 - Persistence models
- `presentation`
 - HTTP controllers
 - Routes
- `bootstrap`
 - Dependency wiring
- `cmd/server`
 - Application entry point

Part 2

- Store interface:

The repository abstraction is defined in:

`internal/domain/ports/link_repository.go`

The interface exposes:

- `Save(link entities.Link)`
- `FindByCode(code string)`
- `FindByURL(url string)`

The use case depends on this abstraction instead of a concrete storage implementation.

- Error handling:

Domain errors are represented using sentinel errors:

- `ErrNotFound`
- `ErrInvalidURL`

Errors are wrapped using `%w` to preserve their identity.

HTTP handlers use `errors.Is()` to map errors into responses:

- `ErrInvalidURL` → `400 Bad Request`
- `ErrNotFound` → `404 Not Found`
- Unknown errors → `500 Internal Server Error`

Error responses are returned as JSON objects containing an `error` field.

Part 3

- Locking choice:

The in-memory store uses `sync.RWMutex`.

Redirect and lookup operations only require read access, allowing multiple concurrent readers.

Write operations acquire exclusive access to prevent race conditions.

- Timeout values:

The HTTP server uses:

- `ReadTimeout`: 5 seconds
- `WriteTimeout`: 10 seconds
- `IdleTimeout`: 60 seconds

These values prevent slow clients from occupying server resources indefinitely.

- Eviction:

No eviction policy is implemented.

The assignment does not require a maximum in-memory capacity, so links remain available while the application process is running.

Part 4

- Storage choice:

PostgreSQL was selected as the persistent storage solution.

GORM is used as the ORM layer to simplify database interaction while keeping the repository abstraction independent from the database.

- Schema and models:

The database contains a `links` table.

Model fields:

- `id`
 - Primary key generated by the database.
- `url`
 - Original long URL.
 - Unique index prevents duplicate URLs.
- `code`
 - Generated short code.
 - Unique index prevents collisions.
- `created_at`
 - Creation timestamp.

Database schema creation is handled using GORM AutoMigrate.

- Crash safety and atomicity:

The application only returns a successful shorten response after the database insertion succeeds.

PostgreSQL provides durable storage, ensuring saved links survive application restarts.

- Created timestamp:

Creation time is stored using Go's `time.Time` type and persisted by GORM in the database `created_at` column.

- Idempotency after restart:

URL-to-code mappings are stored in PostgreSQL.

After restarting the application, a new repository instance can retrieve existing records, so the same normalized URL continues returning the same short code.

Part 5

Part 6